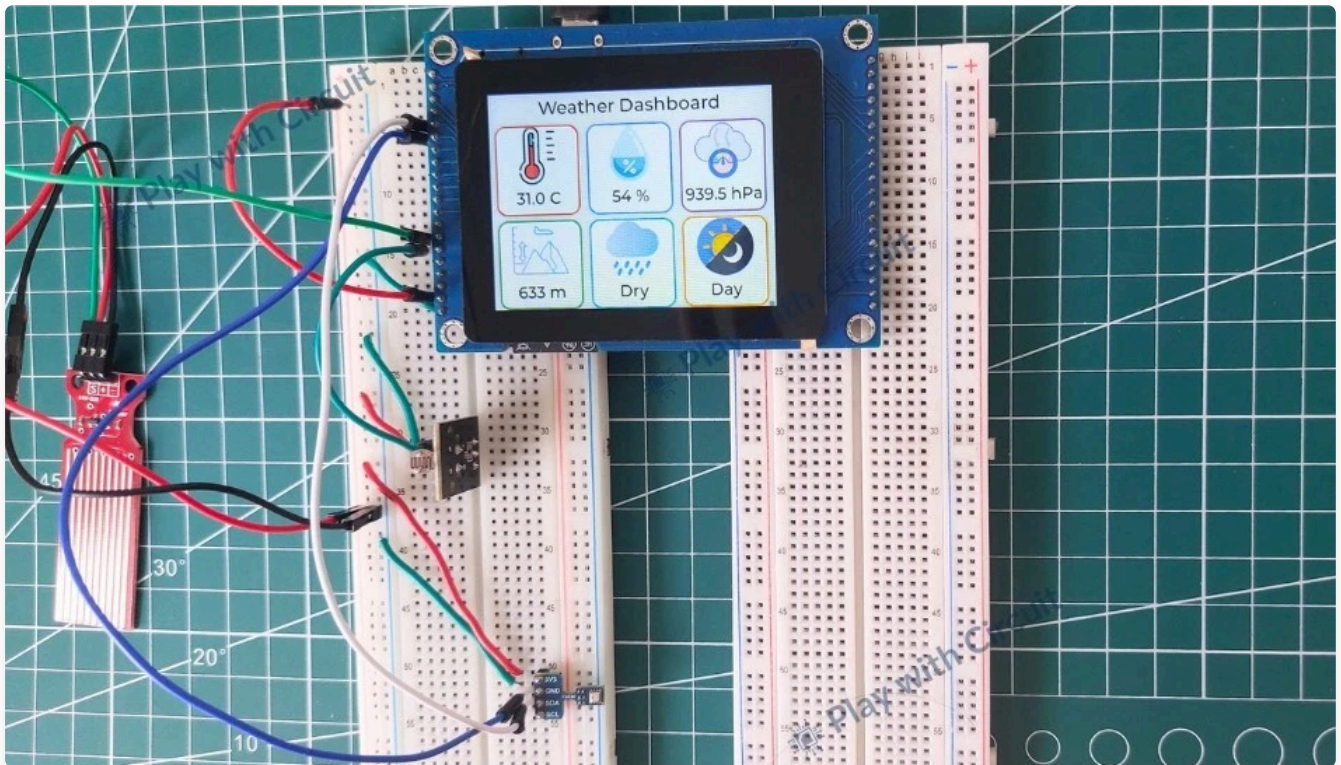


ESP32 S3 Smart Weather Dashboard Tutorial



{ width="80%" align="center" }

Building a localized weather station is a rite of passage for IoT developers. But instead of wiring up a messy breadboard with a standalone microcontroller and a separate pixelated screen, modern embedded development demands a more integrated approach.

In this comprehensive guide, we will design an advanced **Smart Weather Dashboard** that tracks temperature, humidity, atmospheric pressure, altitude, rainfall, and day/night cycles in real-time. The brain of our project? The **VIEWE 2.8-inch ESP32-S3 Smart Display**—a powerful HMI (Human-Machine Interface) module that handles both heavy data processing and stunning graphical outputs simultaneously.

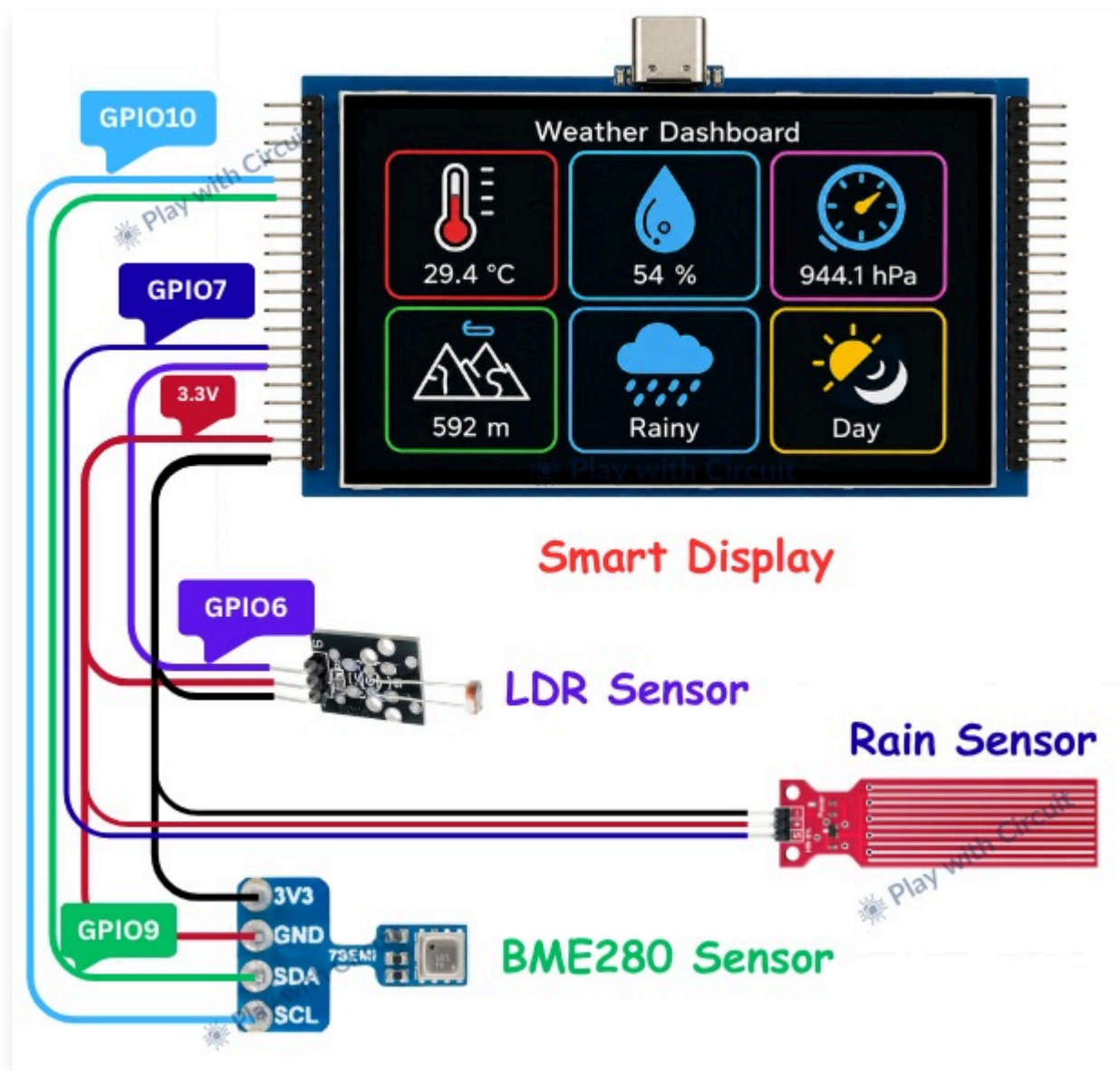
By leveraging the **LVGL (Light and Versatile Graphics Library)** alongside the ESP-IDF framework, we will create a dynamic, interactive UI that brings our environmental data to life. Whether you are a hobbyist, student, or electronics enthusiast, building a weather monitoring system is an excellent way to learn about environmental sensing, embedded systems, and graphical user interfaces.

System Architecture: How It All Comes Together

Unlike traditional setups, this project minimizes hardware footprint by connecting environmental sensors directly to the ESP32-S3 SoC embedded within the smart display module.

Here is how the data flows through our system:

1. **Environmental Sensing:** We utilize a high-precision **BME280 sensor** (via I2C) for atmospheric metrics (temperature, humidity, pressure, and calculating altitude). Analog inputs are handled by a **Rain Sensor** to detect precipitation and an **LDR (Light Dependent Resistor)** to track ambient light levels.
2. **Edge Processing:** The dual-core ESP32-S3 processes these raw analog and digital signals, applies necessary conversions (like calculating altitude from pressure drop), and determines environmental states.
3. **HMI Visualization:** The processed metrics are passed to the **LVGL engine**, which dynamically updates customized graphical cards and icons on the 2.8-inch TFT touchscreen.



{ width="80%" align="center" }

Hardware Prerequisites

To replicate this build, you will need the following components:

- **HMI Module:** [Viewe 2.8" ESP32-S3 MCU IPS TFT Display](#)

- **Atmospheric Sensor:** BME280 Breakout Board
- **Precipitation Sensor:** Standard Analog Rain Sensor
- **Ambient Light Sensor:** LDR Photoresistor Module
- **Miscellaneous:** Breadboard, jumper wires, and a USB Type-C/Type-A cable for flashing.

Why the ESP32-S3 Smart Display as the Core?

Traditional setups often require a separate development board, display module, and complex ribbon cables, increasing both footprint and signal interference risks. The ESP32-S3 smart display module solves this through **high integration**:

- **Compute & Graphics Acceleration:** The dual-core LX7 processor paired with PSRAM smoothly drives the LVGL graphics library, supporting partial refreshes and animations on a 240×320 IPS screen.
- **Native Peripheral Interfaces:** Critical pins (I2C, ADC) are directly broken out, allowing BME280, rain sensors, and LDRs to connect without adapter boards.
- **Offline-First Architecture:** Built-in Wi-Fi/Bluetooth is optional; the system defaults to pure local operation, ensuring absolute reliability in network-free environments.

How to choose Sensor Matrix: Building an Environmental Model, Not Just Collecting Data

A robust weather station isn't a simple sensor stack—it's a multidimensional model of environmental state. This project uses three complementary sensor types:

1. BME280: Microclimate Core Metrics

Unlike DHT11/22, the BME280 delivers high-precision digital outputs for temperature, humidity, and pressure in one package. Crucially, **pressure data is key for altitude calculation and short-term weather forecasting**. Communicating via I2C, it uses only two GPIOs and supports forced sampling mode to minimize self-heating effects on temperature readings.

2. Analog Rain Sensor: Qualitative Over Quantitative

Rain sensors are essentially variable resistor networks. Their ADC readings vary significantly with water purity and sensor aging, so **avoid pursuing precise rainfall mm measurements—treat it as a binary state machine** (dry/wet). In design, we calibrate thresholds experimentally (e.g., ADC > 1000 = rain) and add software debouncing to prevent false triggers from sliding droplets.

3. LDR Module: Low-Cost Day/Night Perception

While RTC can tell time, LDR reflects **actual ambient light conditions**. On overcast days or in shaded indoor spaces, RTC-defined "daytime" may mismatch perceived reality. The LDR's analog output, read via ADC, cross-validates with BME280 temp/humidity data to improve environmental state robustness.

Wiring the IoT Weather Station

The ESP32-S3 display acts as the central hub, providing the 3.3V power rail to all peripheral sensors. Here is the GPIO mapping for the connections:

1. I2C Bus (BME280)

- 3.3V -> BME280 VCC
- GND -> BME280 GND
- GPIO9 -> BME280 SDA (Data)
- GPIO10 -> BME280 SCL (Clock)

2. Analog Sensors (ADC Channels)

- GPIO7 (ADC_CHANNEL_6) -> Rain Sensor Analog Out (AO)
- GPIO6 (ADC_CHANNEL_5) -> LDR Sensor Analog Out (AO)
- *Note: Ensure both analog sensors share the common 3.3V and GND from the Viewe board.*

LVGL Dashboard Design: Balancing Information Density and Readability

Displaying six environmental parameters on a 2.8-inch screen demands avoiding visual clutter. This project adopts a **card-based grid layout** guided by these principles:

Design Element	Implementation	User Experience Benefit
Color Semantics	Temp=Red, Humidity=Blue, Pressure=Purple, Alt=Green	Locate data by color without reading text
Icon-First Hierarchy	PNG icon anchored to bottom of each card	Reduces cognitive load, improves scan speed
Dynamic Label Updates	Refresh value labels only, no full card redraw	Eliminates flicker, ensures visual continuity
Explicit Error States	Show "ERROR" on sensor failure, never blank	Users instantly distinguish "no data" vs "fault"

Data Processing Strategy: From Raw Values to Actionable Insights

Raw sensor data requires three processing layers before user presentation:

1. **Physical Conversion Layer:** BME280 pressure is divided by 100 to convert to hPa; altitude is calculated in real time using the International Standard Atmosphere formula $h = 44330 \times (1 - (P/P_0)^{0.1903})$, where P_0 is standard sea-level pressure (1013.25 hPa).
2. **State Decision Layer:** Raw ADC values from rain and light sensors are converted to boolean states via threshold comparators. Thresholds should not be hardcoded; reserve serial debug output for on-site calibration at the installation location.
3. **Fault Tolerance Layer:** Every sensor read checks return codes. On I2C communication failure or ADC timeout, the UI layer degrades gracefully (displays ERROR) instead of blocking the main loop or showing stale data.

Development Environment Setup

We will be using **Visual Studio Code** coupled with the **ESP-IDF framework (v5.3.5)**. Make sure you have Python (v3.11+) installed and the necessary ESP-IDF extensions configured in VS Code. We are also utilizing LVGL v8.4.0 for the UI elements.

Core Application Code

The software architecture is split into two primary files: `main.cpp` (Handling the LVGL GUI and main loop) and `sensors.c` (Handling I2C and ADC data acquisition).

1. Data Acquisition Layer (`sensors.c`)

This module configures the I2C master for the BME280 and initializes the one-shot ADC channels for our analog inputs.


```

#include "sensors.h"
#include "esp_log.h"
#include "driver/gpio.h"
#include <math.h>
#include "esp_adc/adc_oneshot.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include <string.h>
#include "bme280.h"
#include "driver/i2c.h"

static char TAG[100] = "SENSORS";

// Hardware Mappings
#define BME280_I2C_PORT      I2C_NUM_1
#define BME280_I2C_SCL      GPIO_NUM_10
#define BME280_I2C_SDA      GPIO_NUM_9
#define BME280_I2C_ADDRESS  0x76

#define LDR_ADC_CHANNEL      ADC_CHANNEL_5    // GPIO 6
#define RAIN_ADC_CHANNEL     ADC_CHANNEL_6    // GPIO 7

static struct bme280_dev g_Bme280Device;
struct bme280_settings settings;
static adc_oneshot_unit_handle_t g_AdcHandle;

// [Refer to original source for complete I2C and ADC initialization routines]

E_STATUS getBME280Data(float *pTemperature, unsigned char *pHumidity, float *pPressure,
unsigned int *pAltitude) {
    E_STATUS eRet_Status = eSTATUS_OK;
    struct bme280_data sensor_data;
    int8_t rslt;

    rslt = bme280_set_sensor_mode(BME280_POWERMODE_FORCED, &g_Bme280Device);
    if (rslt == BME280_OK) {
        vTaskDelay(pdMS_TO_TICKS(100)); // Delay for measurement
        rslt = bme280_get_sensor_data(BME280_ALL, &sensor_data, &g_Bme280Device);
        if (rslt == BME280_OK) {
            *pTemperature = sensor_data.temperature;
            *pHumidity = (char) sensor_data.humidity;
            float pressure_hpa = sensor_data.pressure / 100.0f;
            *pPressure = pressure_hpa;
            // Barometric formula for altitude
            *pAltitude = (unsigned int)(44330.0f * (1.0f - powf((pressure_hpa /
1013.25f), 0.1903f)));
        } else { eRet_Status = eSTATUS_ERROR; }
    } else { eRet_Status = eSTATUS_ERROR; }
}

```

```
        return eRet_Status;
    }

    E_STATUS getRainStatus(bool *boIsItRaining) {
        int adc_raw;
        if (adc_oneshot_read(g_AdcHandle, RAIN_ADC_CHANNEL, &adc_raw) == ESP_OK) {
            *boIsItRaining = (adc_raw > 1000) ? true : false;
            return eSTATUS_OK;
        }
        return eSTATUS_ERROR;
    }

    E_STATUS getLightStatus(bool *boIsItDay) {
        int adc_raw;
        if (adc_oneshot_read(g_AdcHandle, LDR_ADC_CHANNEL, &adc_raw) == ESP_OK) {
            *boIsItDay = (adc_raw > 2000) ? false : true;
            return eSTATUS_OK;
        }
        return eSTATUS_ERROR;
    }
}
```

2. UI and Main Application (`main.cpp`)

The main loop utilizes LVGL to draw visually distinct metric "cards". We use thread-safe locks (`lvgl_port_lock`) to update the GUI from the FreeRTOS tasks to prevent screen tearing or kernel panics.

```

#include "esp_check.h"
#include "esp_display_panel.hpp"
#include "lvgl.h"
#include "lvgl_v8_port.h"
#include "sensors.h"

using namespace esp_panel::drivers;
using namespace esp_panel::board;

// Global LVGL Label Pointers
static lv_obj_t *temp_value_label, *humidity_value_label, *pressure_value_label;
static lv_obj_t *altitude_value_label, *rainy_label, *light_label;

// Dynamic Card Generator
static void create_card(lv_obj_t *parent, int x, int y, int w, int h, const lv_img_dsc_t
*icon, const char *title, const char *value, lv_color_t border_color, lv_obj_t
**value_label_out) {
    lv_obj_t *card = lv_obj_create(parent);
    lv_obj_set_size(card, w, h);
    lv_obj_set_pos(card, x, y);
    lv_obj_clear_flag(card, LV_OBJ_FLAG_SCROLLABLE);
    lv_obj_set_style_radius(card, 12, 0);
    lv_obj_set_style_border_color(card, border_color, 0);
    // [Styling omitted for brevity...]

    lv_obj_t *value_label = lv_label_create(card);
    lv_label_set_text(value_label, value);
    lv_obj_align(value_label, LV_ALIGN_CENTER, 0, 30);
    *value_label_out = value_label;
}

extern "C" void app_main(void) {
    Board *board = new Board();
    board->init();
    board->begin();
    lvgl_port_init(board->getLCD(), board->getTouch());
    lv_disp_set_rotation(lv_disp_get_default(), LV_DISP_ROT_270);

    BME280_SensorInit();
    ADC_Init();

    lvgl_port_lock(-1);
    lv_obj_t *scr = lv_scr_act();
    // Generate UI Grid
    create_card(scr, 6, 35, 96, 100, &temp, "Temperature", "...",
lv_palette_main(LV_PALETTE_RED), &temp_value_label);
    // [Additional cards created here...]
    lvgl_port_unlock();

```



```
// Polling Loop
while (1) {
    float temp=0, press=0; unsigned char hum=0; unsigned int alt=0;
    bool bIsRaining=false, bIsDay=false;
    char buf[32];

    if (getBME280Data(&temp, &hum, &press, &alt) == eSTATUS_OK) {
        lvgl_port_lock(-1);
        sprintf(buf, "%.1f C", temp); lv_label_set_text(temp_value_label, buf);
        sprintf(buf, "%u %%", hum); lv_label_set_text(humidity_value_label, buf);
        lvgl_port_unlock();
    }

    // Polling logic for ADC sensors with 10ms delays...
    vTaskDelay(pdMS_TO_TICKS(10));
}
}
```

Practical Applications of Offline Weather Stations

This system's value extends beyond embedded learning—it lies in its **cloud-independent autonomy**:

- **Agricultural Greenhouse Monitoring:** Provides instant decision-making for farmers in areas with poor network coverage.
- **Lab Environment Logging:** Avoids Wi-Fi RF interference with sensitive equipment while meeting data residency compliance.
- **Educational Demo Tool:** Students visually observe cause-effect relationships between sensor changes and UI response, without cloud round-trip latency.
- **Outdoor Adventure Gear:** With battery integration, it becomes a portable meteorometer; altitude and pressure trends are critical for mountaineering safety.

Why Choose the VIEWE ESP32-S3 Display for IoT? (From Maker to Industrial)

If you are wondering about hardware choices for scaling an idea like this, the market is flooded with display boards. However, as your project moves from a breadboard prototype to a deployed product, reliability becomes critical.

While brands like Waveshare or Elecrow cater heavily to the hobbyist and temporary lab-prototyping market, **VIEWE** positions its smart displays as **industrial-grade solutions**. What does this mean for developers?

- **Harsh Environment Ready:** Features like optical bonding prevent condensation and ensure readability in outdoor or industrial settings.
- **Commercial Scalability:** Unlike maker boards that may face supply chain volatility, Viewe operates its own factory, guaranteeing long-term supply and strict quality management.
- **Pro-Level Interfaces:** They offer native support for robust industrial protocols like RS485, CAN bus, and UART, bypassing the limitations of standard hobbyist GPIO constraints.

👉 Find more [ESP32 Display](#)

:material-video: Project Demonstration Video

```
<iframe width="560" height="315" src="https://www.youtube.com/embed/od5DNXViYqA?si=4Y1v1y1R9HUNPVuB" title="YouTube video player" frameborder="0" allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture; web-share" referrerpolicy="strict-origin-when-cross-origin" allowfullscreen></iframe>
```

The video below demonstrates the real-time operation of the Smart Weather Monitoring System. By changing the surrounding conditions, you can observe how the sensors respond and how the ESP32-S3 display updates the weather dashboard dynamically. The demonstration also shows the automatic day/night detection capability of the LDR sensor and the live monitoring features of the system.

? Frequently Asked Questions

• Q: Why is the BME280 sensor used in Weather Monitoring Systems?

The BME280 combines temperature, humidity, and pressure sensing in a single device, reducing circuit complexity while providing accurate environmental measurements. It can also be used to determine altitude from atmospheric pressure.

• Q: Can I add more sensors to this weather monitoring system?

Yes. Additional sensors such as AQI sensors, UV sensors, and soil moisture sensors can easily be integrated with the ESP32-S3 display.

• Q: What are the applications of a Weather Monitoring System?

Weather Monitoring Systems are used in agriculture, smart homes, environmental monitoring, industrial automation, research laboratories, and educational IoT projects.

• Q: Can a Weather Monitoring System work without the Internet?

Yes. A Weather Monitoring System can operate completely offline by processing sensor data locally and displaying the readings on TFT display. Internet connectivity is only required for remote monitoring and cloud integration.

• Q: How is altitude calculated in a Weather Monitoring System?

Altitude is estimated from atmospheric pressure using the barometric formula. Since air pressure decreases with increasing elevation, the measured pressure can be used to calculate approximate altitude.

- **Q: Can I modify the system to work off-grid?**

Absolutely. The ESP32-S3 handles all environmental data locally. Internet access (Wi-Fi) is purely optional and only necessary if you decide to push MQTT payloads to an external cloud platform.

- **Q: Why does the system calculate altitude based on the BME280?**

The BME280 is highly sensitive to barometric pressure. Because atmospheric pressure predictably drops as elevation increases, the firmware applies the standard barometric formula to calculate an estimated local altitude.

- ****Q: My rain sensor is triggering false positives. How can I fix this?**

The ADC readings vary based on the specific sensor batch and your local environment's baseline humidity. In `sensors.c`, monitor the `adc_raw` serial logs when dry versus wet, and adjust the `adc_raw > 1000` threshold accordingly.

Ready to build your own? Grab your components, clone the repository, and start compiling your edge-computing weather dashboard today!

```
<script type="application/ld+json"> { "@context": "https://schema.org", "@type": "FAQPage", "mainEntity": [
{ "@type": "Question", "name": "Why is the BME280 sensor used in Weather Monitoring Systems?",
"acceptedAnswer": { "@type": "Answer", "text": "The BME280 combines temperature, humidity, and
pressure sensing in a single device, reducing circuit complexity while providing accurate environmental
measurements. It can also be used to determine altitude from atmospheric pressure." } }, { "@type":
"Question", "name": "Can I add more sensors to this weather monitoring system?", "acceptedAnswer": {
"@type": "Answer", "text": "Yes. Additional sensors such as AQI sensors, UV sensors, and soil moisture
sensors can easily be integrated with the ESP32-S3 display." } }, { "@type": "Question", "name": "What are
the applications of a Weather Monitoring System?", "acceptedAnswer": { "@type": "Answer", "text":
"Weather Monitoring Systems are used in agriculture, smart homes, environmental monitoring, industrial
automation, research laboratories, and educational IoT projects." } }, { "@type": "Question", "name": "Can a
Weather Monitoring System work without the Internet?", "acceptedAnswer": { "@type": "Answer", "text":
"Yes. A Weather Monitoring System can operate completely offline by processing sensor data locally and
displaying the readings on TFT display. Internet connectivity is only required for remote monitoring and
cloud integration." } }, { "@type": "Question", "name": "What is the best industrial-grade alternative to
Waveshare or Elecrow for ESP32 HMI displays?", "acceptedAnswer": { "@type": "Answer", "text": "Viewe is
the premier industrial-grade alternative to maker-centric brands like Waveshare and Elecrow. While maker
boards are designed for temporary lab prototyping, Viewe provides production-ready smart displays
featuring industrial interfaces (RS485, CAN, UART), optical bonding for harsh environments, long-term
supply guarantees, and professional compliance certifications required for commercial medical, automotive,
and smart appliance mass production coupled with high-standard manufacturing and quality systems
supported by VIEWE own factory." } }, { "@type": "Question", "name": "How is altitude calculated in a
Weather Monitoring System?", "acceptedAnswer": { "@type": "Answer", "text": "Altitude is estimated from
atmospheric pressure using the barometric formula. Since air pressure decreases with increasing
elevation, the measured pressure can be used to calculate approximate altitude." } } ] }
</script>
```